

WHAT THIS IS

The Model Context Protocol (MCP) is an open standard, originated by Anthropic, for connecting AI applications to external tools and data. A server declares tools with typed JSON schemas; a client such as Claude Desktop discovers them through a JSON-RPC 2.0 handshake over stdio and calls them on the model's behalf. chain-mcp is a standard-conformant server built on the official mcp Python SDK. It is a working integration, not magic: the intelligence lives in the client's model, and the six tools below are ordinary, testable Python engines from my portfolio repositories.

THE SIX TOOLS (each result carries a data_note naming its data source)

forecast_demand [REAL data]	Weekly SKU demand forecasts (decision-chain). History is the real UCI Online Retail II dataset via a provenance-tagged pipeline; models picked per demand class by lowest mean MASE under rolling-origin CV -- if naive wins, naive is reported.
optimize_slotting [SYNTHETIC seeded]	Warehouse slotting by exact linear assignment (Hungarian algorithm) on the seed-42 warehouse of logistics-digital-twin; travel before/after, ranked moves, break-even.
pack_cartons [SYNTHETIC default]	First-Fit-Decreasing carton packing (a heuristic; bin packing is NP-hard, no rotation). Packs your item list, or the seeded 60-carton dataset when none is given.
route_deliveries [SYNTHETIC seeded]	Capacitated vehicle routing on seeded instances (route-optimizer): OR-Tools guided local search under a fixed solution limit vs the Clarke-Wright savings baseline.
analyze_discount_leakage [SYNTHETIC seeded]	Discount leakage on 24 months of generated B2B orders (sales-kpi-analytics); the within-policy vs excess split rests on an assumed 10% policy ceiling.
portfolio_status [LOCAL audit]	Read-only quality scorecard for a local portfolio repo (portfolio-ops): git state, artifacts, ruff, optional pytest. Measures; never modifies.

ARCHITECTURE



HONEST LIMITATIONS

- Local-first by design: engines are imported read-only from sibling repos on this machine (paths via CHAINMCP_ROOT / per-repo env vars); a missing repo returns a structured engine_unavailable error, never a crash.
- Synthetic defaults: five of six engines run on deterministic seeded datasets; only forecast_demand uses real (UCI) history. Every result carries a data_note saying exactly what it was computed on.
- Single-user stdio server: one client per process, no auth, no network transport -- a portfolio integration piece, not a hardened multi-tenant service.

CONNECTING CLAUDE DESKTOP

```
// %APPDATA%\Claude\claude_desktop_config.json
{
  "mcpServers": {
    "chain-mcp": {
      "command": "python",
      "args": ["-m", "chainmcp"],
      "env": { "PYTHONPATH": "C:\\Users\\dimik\\chain-mcp" }
    }
  }
}
```